

WEB API & ANGULAR CON VISUAL STUDIO .NET CORE

Capítulo 6: Seguridad

Four empty circles arranged vertically on the right side of the slide.



OBJETIVOS

- Autenticar una API.
- Identificar el formato JWT para implementar autenticación.
- Configurar CORS.





AGENDA

1. Autenticación
2. JWT
3. Implementando JWT
4. Autorización basada en claims via JWT
5. Configurar filtros de autenticación en los controladores
6. Configuración de CORS

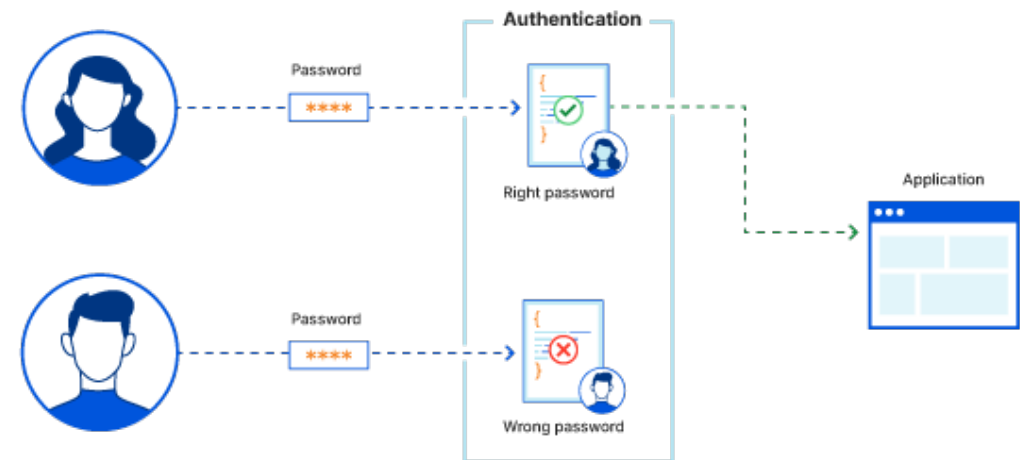




1. AUTENTICACIÓN

La autenticación es el proceso de determinar la identidad de un usuario.

La autorización es el proceso de determinar si un usuario o sistema tiene acceso a un recurso.





2. JWT

Uno de los mecanismos más conocidos para implementar autenticación en las aplicaciones es mediante Json Web Token(JWT). Es un estándar abierto (RFC 7519) que define una forma compacta y autónoma de transmitir información de forma segura entre las partes como un objeto JSON. Esta información se puede verificar y confiar porque está firmada digitalmente.

Autenticación es el escenario más común para usar JWT. Una vez que el usuario haya iniciado sesión, cada solicitud posterior incluirá el JWT, lo que le permitirá acceder a rutas, servicios y recursos que están permitidos con ese token.





2. JWT

¿Estructura de un JSON Web Tokens?

En su forma compacta, los tokens web JSON constan de tres partes separadas por puntos (.), que son:

Encabezado: El encabezado generalmente consta de dos partes: el tipo de token, que es JWT, y el algoritmo de firma que se utiliza, como HMAC SHA256 o RSA.

- Cuerpo: Información. Por ejemplo, datos del usuario.
- Firma: Para crear la parte de la firma, debe tomar el encabezado codificado, el cuerpo, un secreto, el algoritmo especificado en el encabezado y se firma.

Por lo tanto, un JWT suele tener el siguiente aspecto.

xxxxx.yyyyyy.zzzzz

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmlxdWVfbmFtZSI6IkpndmlldmllciIsInJvbGUOiJhZG1pbilzIm5iZiI6MTYzODM5MDQ4OSwiZXhwIjoxNjM4Mzk0MDgyLCJpYXQiOjE2Mzg5OTA0ODI9.rsWGh6o32cRzoMT7JbPTn9Yiv0WMw_HY-



3. IMPLEMENTANDO JWT

Instalar las siguientes dependencias:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Configurar en el método ConfigureServices del archivo Startup lo siguiente:

```
services.AddAuthentication().AddJwtBearer
```

Configurar en el archivo Console.cs lo siguiente:

```
UseAuthentication()
```

```
Antes de UseAuthorization()
```



4. AUTORIZACIÓN BASADA EN CLAIMS VIA JWT



Una vez identificado el usuario, .NET nos ofrece múltiples formas de validar sus permisos, dentro de las cuales podemos destacar:

Autorización basada en roles: Al crearse una entidad, ésta puede pertenecer a uno o varios roles, los cuales se usan para validar el acceso del usuario a los servicios.

```
claims.Add(new Claim(ClaimTypes.Role, role.Name));
```

```
{
  "sub": "rstrahl",
  "jti": "25544595-31ad-4708-8191-67579ad1834b",
  "username": "rstrahl",
  "UserState": "||rstrahl|False|-1|null|6cb34183",
  "role": [
    "User",
    "Administrator"
  ],
  "exp": 1615363222,
  "iss": "https://idVerify.com",
  "aud": "https://idVerify.com"
}
```





5. CONFIGURAR FILTROS DE AUTENTICACIÓN EN LOS CONTROLADORES

- AllowAnonymos: Permitir acceso anónimo, sin ninguna autenticación;
- Authorize: Para acceder es necesario estar autenticado;
- Authorize(Roles="role1,role2,role3"): Además de estar autenticado, el rol asociado al usuario debe estar en la lista.

[Authorize]

[Route("api/[controller]")]

[ApiController]

1 referência

public class CategoriaController : ControllerBase

{

private readonly IMediator mediator;

```
[HttpGet]
```

```
[Route("employee")]
```

```
[Authorize(Roles = "employee,manager")]
```

```
public string Employee() => "Funcionário";
```



6. CONFIGURACIÓN DE CORS – CROSS-ORIGIN RESOURCE SHARING

Cross-Origin Resource Sharing (CORS) , es un mecanismo basado en encabezados HTTP que permite a un servidor indicar cualquier origen (dominio, esquema o puerto) distinto del suyo desde el cual un navegador debería permitir la carga de recursos.

Adicionar el siguiente código:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy(MyAllowSpecificOrigins,
        policy =>
        {
            policy.WithOrigins("http://example.com",
                               "http://www.contoso.com")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});
```

```
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseCors();
app.UseAuthorization();

app.MapControllers();

app.Run();
```





LABORATORIO N° 1: ADICIONAR SEGURIDAD BASADA EN TOKEN PARA EL SERVICIO REST IMPLEMENTADO DURANTE LAS CLASES

En este laboratorio, podrás conocer la implementación de seguridad basada en JWT.





LABORATORIO Nº 2: ADICIONAR CORS PARA EL SERVICIO REST IMPLEMENTADO DURANTE LAS CLASES

En este laboratorio, podrás conocer cómo implementar CORS en un proyecto Net Core





TAREA Nº 1: CONFIGURAR UN JWT PARA SOPORTAR AUTORIZACIÓN BASADO EN ROLES

Al finalizar la tarea, lograrás:

- Configurar la autenticación basado en roles.





LECTURAS ADICIONALES

Para obtener información adicional, puede consultar:

- Microsoft Build (30 de noviembre de 2023). *Descripción general de la autenticación de ASP.NET Core*. <https://learn.microsoft.com/pt-br/aspnet/core/security/authentication/?view=aspnetcore-8.0>
- <https://weblog.west-wind.com/posts/2021/Mar/09/Role-based-JWT-Tokens-in-ASPNET-Core>
- Microsoft Build (01 de febrero de 2024). *Enable Cross-Origin Requests (CORS) in ASP.NET Core*. <https://learn.microsoft.com/en-us/aspnet/core/security/cors?view=aspnetcore-8.0>





RESUMEN

- La autenticación es el proceso de determinar la identidad de un usuario.
- Para la implementar autenticación, se utiliza el formato JWT, el cual consta de tres pares separadas por puntos (.), que son: encabezado, cuerpo y firma.
- Cross-Origin Resource Sharing (CORS), permite a un servidor indicar cualquier origen distinto del suyo desde el cual un navegador debería permitir la carga de recursos



